

# Тема урока: цикл for

## Цикл for

Одно из преимуществ компьютеров перед людьми - способность повторять одни и те же действия многократно, быстро и совсем не утомляясь 🖥️.

В Python существует две основных разновидности циклов:

- циклы, повторяющиеся определенное количество раз ( `for` , счетные циклы, **counting loops**);
- циклы, повторяющиеся до наступления определенного события ( `while` , условные циклы, **conditional loops**).

Цикл `for` замечательно работает, если мы заранее знаем, сколько повторений (итераций) нам требуется сделать.

Рассмотрим код, который распечатает 10 раз слово "Привет":

```
for i in range(10):  
    print('Привет')
```

Структура цикла `for` в Python выглядит так:

```
for название_переменной_цикла in range(количество_повторений):  
    блок кода
```

Двоеточие ( `:` ) в конце строки с инструкцией `for` сообщает интерпретатору Python, что дальше находится **блок команд**. В блок команд входят все строки, расположенные с отступом от строки с инструкцией `for` , вплоть до следующей строки без отступа.



Блок команд, который выполняется в цикле `for`, называется **телом цикла**.

В предыдущих уроках мы считывали несколько чисел при помощи нескольких команд `input()`. С помощью цикла `for` можно считывать и обрабатывать сколько угодно чисел.

Рассмотрим следующий программный код:

```
for i in range(5):  
    num = int(input())  
    print("Квадрат вашего числа равен:", num * num)  
  
print("Цикл завершен")
```

Такая программа считывает 5 чисел и выводит на экран их квадраты вместе с поясняющей надписью. Поскольку вторая и третья строки выделены отступом, Python считает, что это тело цикла, которое выполняется 5 раз. Четвертая строка не содержит отступа, поэтому не является частью цикла и будет выполнена всего один раз, после того как цикл завершится.

## Примеры использования цикла for

Рассмотрим следующий программный код:

```
print("A")  
print("B")  
  
for i in range(5):  
    print("C")  
    print("D")  
  
print("E")
```

Результатом выполнения такой программы будут строки

```
A  
B  
C  
D  
C  
D  
C  
D  
C  
D  
C  
D  
E
```

То есть сначала программа распечатает символы **A** и **B**, затем символы **C** и **D** пять раз, а затем распечатает символ **E** один раз. Тело цикла состоит из двух строк: пятой и шестой; именно они и будут повторяться.

В программе может быть сколько угодно циклов. Например, если мы хотим, чтобы сначала 5 раз был распечатан символ **C**, а затем 5 раз символ **D**, мы можем использовать 2 цикла:

```
print('A')
print('B')

for i in range(5):
    print('C')

for i in range(5):
    print('D')

print('E')
```

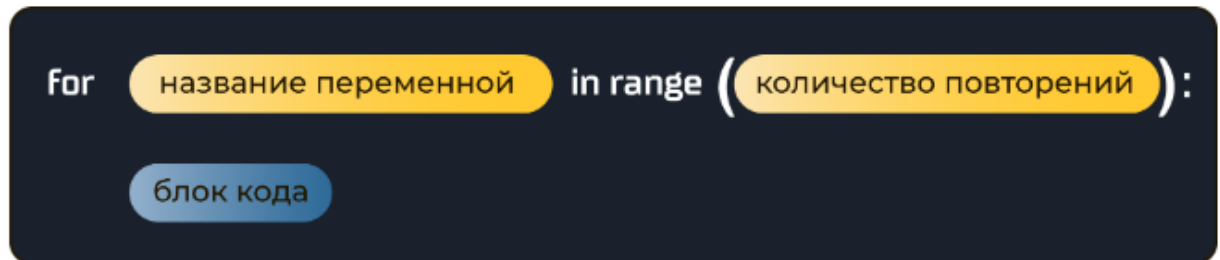
Результатом выполнения такой программы будут строки:

```
A
B
C
C
C
C
C
C
D
D
D
D
D
E
```

## Примечания

**Примечание 1.** Однократное выполнение тела цикла называется **итерацией цикла**.

**Примечание 2.** Графическое представление цикла `for` имеет вид:



**Примечание 3.** Напомним, что **блоком кода** называют объединенные друг с другом строки. Они всегда связаны с определенной частью программы (например, с инструкцией `if` или `for`). В Python блоки кода формируются при помощи **отступов**:

```
1  for i in range(5):
2      print("C")
3      print("D")
4
```

**Примечание 4.** Слово `for` пишется маленькими буквами, первая строка должна заканчиваться двоеточием, а тело цикла должно быть выделено отступом.

## Переменная цикла

Давайте еще раз взглянем на базовую структуру цикла `for`:

```
for название_переменной_цикла in range(количество_повторений):
    блок кода
```

Не совсем понятно, для чего нужна и как работает переменная цикла.

Рассмотрим следующий код:

```
for i in range(10):
    print(i)
```

Результатом выполнения такого кода будет:

```
0
1
2
3
4
5
6
7
8
9
```

Когда цикл впервые начинает работу, Python устанавливает значение переменной цикла `i = 0`. Каждый раз, когда мы повторяем тело цикла, Python увеличивает значение переменной на 1.



Почему большинство программистов начинают цикл с 0, а не с 1? Раньше некоторые начинали цикл с 1, а некоторые с 0. Те и другие приводили весьма изощренные аргументы, споря о том, какой способ лучше. Но в конце концов победили сторонники второго варианта. С

тех пор большинство начинает циклы с 0. В частности, в Python цикл `for` начинается с 0, однако в будущих уроках вы узнаете, как это изменить.

Поскольку переменная цикла `i` увеличивается на 1 каждый раз, то ее можно использовать для отслеживания номера итерации, на которой мы находимся в циклическом процессе.

Рассмотрим следующий код:

```
for i in range(10):
    print(i, '-- Привет')
```

Результатом выполнения такого кода будет:

```
0 -- Привет
1 -- Привет
2 -- Привет
3 -- Привет
4 -- Привет
5 -- Привет
6 -- Привет
7 -- Привет
8 -- Привет
9 -- Привет
```

Если мы хотим начать с 1, то можем написать код:

```
for i in range(10):  
    print(i + 1, '-- Привет')
```

Результатом выполнения такого кода будет:

```
1 -- Привет  
2 -- Привет  
3 -- Привет  
4 -- Привет  
5 -- Привет  
6 -- Привет  
7 -- Привет  
8 -- Привет  
9 -- Привет  
10 -- Привет
```

Обратите внимание, за счет выражения `i + 1`, мы начинаем вывод с 1, а не с 0.

## Имена переменных цикла

Ранее говорилось, что имена переменных должны носить осмысленный характер и описывать их назначение. Однако для переменных цикла иногда делаются исключения. В программировании для переменных цикла **обычно** используют буквы `i`, `j`, `k`.

Следующие две программы абсолютно одинаковые: в первой программе переменная цикла имеет название `i`, во второй программе – `number`:

```
for i in range(10):  
    print(i)
```

```
for number in range(10):  
    print(number)
```

Результатом выполнения обеих программ будет:

```
0  
1  
2  
3  
4  
5  
6  
7  
8  
9
```



Почему для переменной циклов зарезервированы буквы `i`, `j`, `k`? Дело в том, что раньше программы использовались для математических расчетов, а в математике буквы `a`, `b`, `c` и `x`, `y`, `z` уже зарезервированы для других целей. Поэтому программисты выбрали для этой цели переменные `i`, `j`, `k`, и это стало общепринятой практикой.

Бывают ситуации, когда переменная цикла не используется в теле цикла. В таком случае, вместо того, чтобы давать ей имя, мы можем указать символ нижнего подчеркивания `_`:

```
for _ in range(5):  
    print('Python - awesome!')
```

Результатом выполнения такого кода будет:

```
Python - awesome!  
Python - awesome!  
Python - awesome!  
Python - awesome!  
Python - awesome!
```



Если переменная цикла не используется в теле цикла, то указывайте вместо неё символ нижнего подчеркивания `_`.

## Примечания

**Примечание.** Следует помнить, что правая граница цикла в Python всегда не включительна. Таким образом следующий код:

```
for i in range(5):  
    print(i)
```

распечатает числа от 0 до 4:

```
0  
1  
2  
3  
4
```

Если требуется распечатать числа от 1 до 5, то мы пишем код:

```
for i in range(5):  
    print(i + 1)
```